

Functional Dependencies and Normalization for Relational Databases

Relational Database Designs

- There are many different relational database "designs" (schemas) that can be used to store the relevant mini-world information.
- Can one schema be much better than another?



EXAMPLE: SHIPMENT OF PARTS

Ship(S#, sname, status, city, P#, pname, colour, weight, qty, date)

S#	sname	status	city	P#	pname	colour	weight	qty	date
S1	Smith	20	London	P1	Nut	Red	12	200	980620
S1	Smith	20	London	P1	Nut	Red	12	700	980625
S2	Jones	10	Paris	P3	Screw	Blue	17	400	980620
S2	Jones	10	Paris	P5	Bolt	Green	17	300	980620
S2	Jones	10	Paris	P2	Screw	Red	14	200	980621
S3	Clark	20	Rome	P1	Nut	Red	12	300	980612
S3	Clark	20	Rome	P6	Cog	Red	19	600	980612
S4	Blake	30	Athens	P4	Cam	Blue	12	200	980619
S4	Blake	30	Athens	P1	Nut	Red	12	300	980619
S4	Blake	30	Athens	P3	Screw	Blue	17	100	980620
S5	Alex	10	Paris	P1	Nut	Red	12	250	980626

Above table contains information regarding shipments of parts from suppliers.

- A supplier belongs to a particular city and a status is associated with each city.
- A part has a name, color, quantity and weight and is shipped on particular date

EXAMPLE: SHIPMENT OF PARTS

Ship(S#, sname, status, city, P#, pname, colour, weight, qty, date)

S#	sname	status	city	P#	pname	colour	weight	qty	date
S1	Smith	20	London	P1	Nut	Red	12	200	980620
S1	Smith	20	London	P1	Nut	Red	12	700	980625
S2	Jones	10	Paris	P3	Screw	Blue	17	400	980620
S2	Jones	10	Paris	P5	Bolt	Green	17	300	980620
S2	Jones	10	Paris	P2	Screw	Red	14	200	980621
S3	Clark	20	Rome	P1	Nut	Red	12	300	980612
S3	Clark	20	Rome	P6	Cog	Red	19	600	980612
S4	Blake	30	Athens	P4	Cam	Blue	12	200	980619
S4	Blake	30	Athens	P1	Nut	Red	12	300	980619
S4	Blake	30	Athens	P3	Screw	Blue	17	100	980620
S5	Alex	10	Paris	P1	Nut	Red	12	250	980626

Problems ??

- Redundant Data
- Inability to represent certain information
- Loss of information



Design Anomalies

Ship(S#, sname, status, city, P#, pname, colour, weight, qty, date)

- This Apply relation exhibits three types of anomalies:
 - Redundancy
 - Update Anomaly
 - Deletion Anomaly
- Good designs:
 - No anomalies
 - Can reconstruct all original information



EXAMPLE: SHIPMENT OF PARTS

The solution is to decompose Ship into three relations Shipment, Supplier, and Parts

Supplier(S#, sname, status, city)

Parts(P#, pname, colour, weight)

Shipment(S#, P#, qty, date)

S#	sname	status	city
S1	Smith	20	London
S2	Jones	10	Paris
S3	Clark	20	Rome
S4	Blake	30	Athens
S5	Alex	10	Paris

P#	pname	colour	weight
P1	Nut	Red	12
P2	Screw	Red	14
P3	Screw	Blue	17
P4	Cam	Blue	12
P5	Bolt	Green	17
P6	Cog	Red	19

S#	P#	qty	date
S1	P1	200	980620
S1	P1	700	980625
S2	P3	400	980620
S2	P5	300	980620
S2	P2	200	980621
S3	P1	300	980612
S3	P6	600	980612
S4	P4	200	980619
S4	P1	300	980619
S4	P3	100	980620
S5	P1	250	980626



EXAMPLE: SHIPMENT OF PARTS

The solution is to decompose Supplier into 2 relations Supplier and Status

Supplier(S#, sname, city) Status(city, status)

Parts(P#, pname, colour, weight)

Shipment(S#, P#, qty, date)

S#	sname	city
S1	Smith	London
S2	Jones	Paris
S3	Clark	Rome
S4	Blake	Athens
S5	Alex	Paris

city	status
London	20
Paris	10
Rome	20
Athens	30

P#	pname	colour	weight
P1	Nut	Red	12
P2	Screw	Red	14
P3	Screw	Blue	17
P4	Cam	Blue	12
P5	Bolt	Green	17
P6	Cog	Red	19

S#	P#	qty	date
S1	P1	200	980620
S1	P1	700	980625
S2	P3	400	980620
S2	P5	300	980620
S2	P2	200	980621
S3	P1	300	980612
S3	P6	600	980612
S4	P4	200	980619
S4	P1	300	980619
S4	P3	100	980620
S5	P1	250	980626



Informal Design Guidelines for Relational Databases

- We first discuss informal guidelines for good relational design
- Then we discuss formal concepts of functional dependencies and normal forms



INFORMAL DESIGN GUIDELINES FOR RELATIONAL DATABASES

NULL VALUES IN TUPLES

- Relations should have few NULL values
- Attributes that are NULL frequently could be placed in separate relations (with the primary key)
- Reasons for nulls:
 - Attribute not applicable or invalid
 - Attribute value unknown (may exist)
 - Value known to exist, but unavailable



SEMANTICS OF THE RELATION ATTRIBUTES

- *Each tuple in a relation should represent one entity or relationship instance.*

S#	sname	status	city	P#	pname	colour	weight	qty	date
S1	Smith	20	London	P1	Nut	Red	12	200	980620
S1	Smith	20	London	P1	Nut	Red	12	700	980625
S2	Jones	10	Paris	P3	Screw	Blue	17	400	980620
S2	Jones	10	Paris	P5	Bolt	Green	17	300	980620
S2	Jones	10	Paris	P2	Screw	Red	14	200	980621
S3	Clark	20	Rome	P1	Nut	Red	12	300	980612
S3	Clark	20	Rome	P6	Cog	Red	19	600	980612
S4	Blake	30	Athens	P4	Cam	Blue	12	200	980619
S4	Blake	30	Athens	P1	Nut	Red	12	300	980619
S4	Blake	30	Athens	P3	Screw	Blue	17	100	980620
S5	Alex	10	Paris	P1	Nut	Red	12	250	980626

- Attributes of different entities should not be mixed in the same relation
- Only foreign keys should be used to refer to other entities

RELATION DECOMPOSITION

- Relation decomposition can be dangerous we can lose information.
- Loss-less join decomposition
- If we decompose a relation R into smaller relations, the join of those relations should return R , not more, not less.



EXAMPLE: RELATION DECOMPOSITION

<i>SID</i>	<i>name</i>	<i>email</i>	<i>CID</i>	<i>grade</i>
142	Bart	bart@fox.com	CPS116	B-
142	Bart	bart@fox.com	CPS114	B
123	Milhouse	milhouse@fox.com	CPS116	B+
857	Lisa	lisa@fox.com	CPS116	A+
857	Lisa	lisa@fox.com	CPS130	A+
456	Ralph	ralph@fox.com	CPS114	C
...	...	...	...	...

Redundant Information

- Eliminates redundancy
- To get back to the original relation use join

<i>SID</i>	<i>name</i>	<i>email</i>	<i>CID</i>	<i>grade</i>
...	...	...	...	...

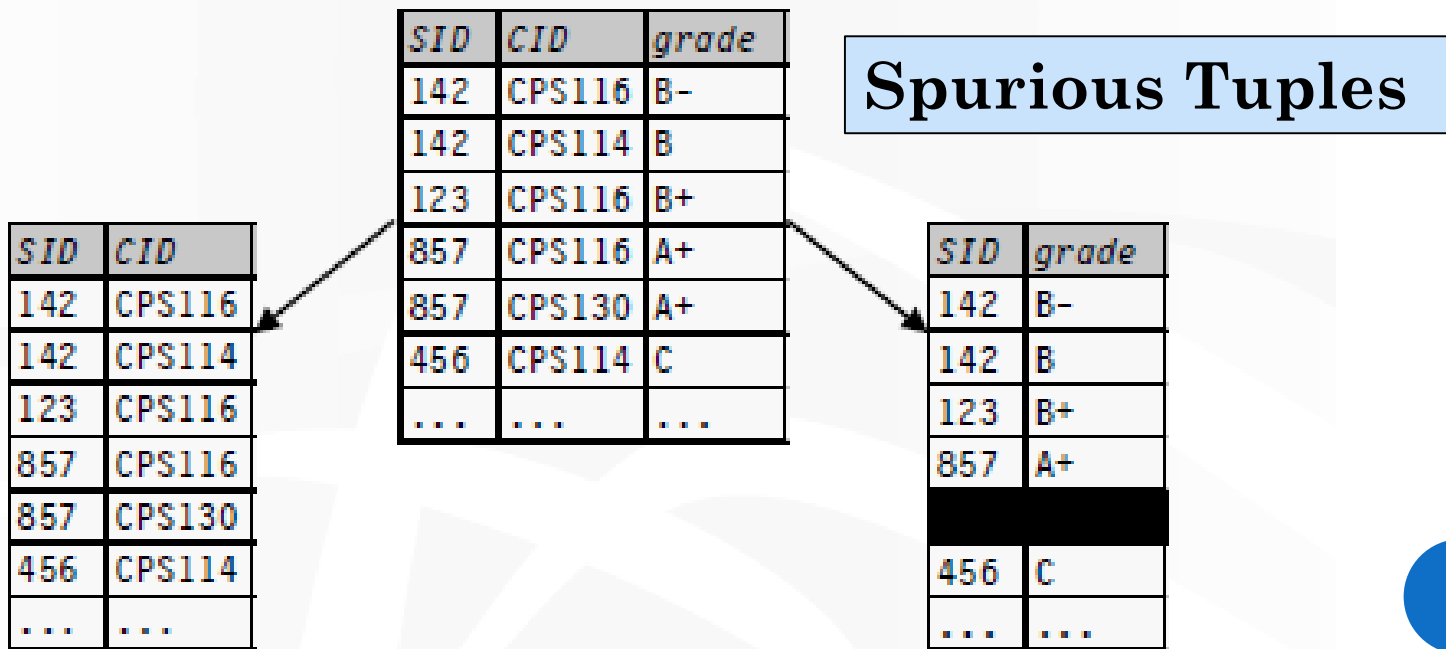
<i>SID</i>	<i>name</i>	<i>email</i>
142	Bart	bart@fox.com
123	Milhouse	milhouse@fox.com
857	Lisa	lisa@fox.com
456	Ralph	ralph@fox.com
...	...	...

<i>SID</i>	<i>CID</i>	<i>grade</i>
142	CPS116	B-
142	CPS114	B
123	CPS116	B+
857	CPS116	A+
857	CPS130	A+
456	CPS114	C
...	...	...

RELATION DECOMPOSITION

BAD DECOMPOSITION

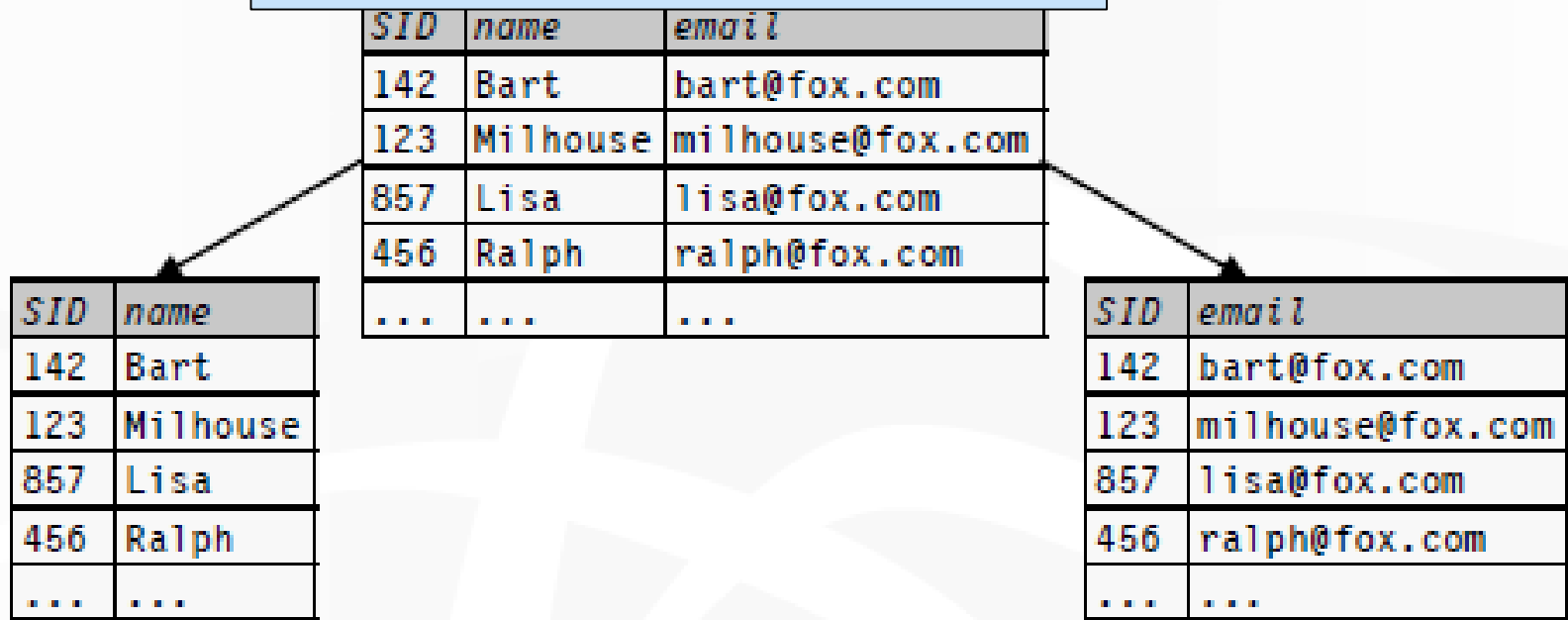
- Association between *CID* and *grade* is lost
- Join returns more rows than the original relation



RELATION DECOMPOSITION

- Join returns the original relation
- Unnecessary: no redundancy is removed, and now *SID* is stored twice!

Unnecessary decomposition



Functional Dependencies

- How do we tell if a design is bad?

<i>SID</i>	<i>name</i>	<i>CID</i>
142	Bart	CPS116
142	Bart	CPS114
857	Lisa	CPS116
857	Lisa	CPS130
...	...	...

- This design has redundancy and update anomalies
- How about a systematic approach to detecting and removing redundancy in designs?
 - Dependencies, decompositions, and normal forms



FUNCTIONAL DEPENDENCIES

- Functional dependencies play an important role in database design.
- They describe relationships between attributes.
- FDs are derived from the real-world constraints on the attributes



FUNCTIONAL DEPENDENCIES

- A functional dependency (FD) has the form $X \rightarrow Y$, where X and Y are sets of attributes in a relation R
- $X \rightarrow Y$ means that whenever two tuples in R agree on all the attributes in X , they must also agree on all attributes in Y

X	Y	Z
a	b	c
a	b	$?$
...	...	...

Must be b

Could be anything

For any two tuples t_1 and t_2 in any relation instance $r(R)$:
If $t_1[X] = t_2[X]$, then $t_1[Y] = t_2[Y]$

- We say X functionally determines Y
- $X \rightarrow Y$ in R specifies a *constraint* on all relation instances $r(R)$

EXAMPLE: STUDENT GRADE INFO

- *StudentGrade* (*SID*, *name*, *email*, *CID*, *grade*)
 - $SID \rightarrow name, email$
 - $email \rightarrow SID$
 - $SID, CID \rightarrow grade$
- Not a good design



EXAMPLE: ADDRESS

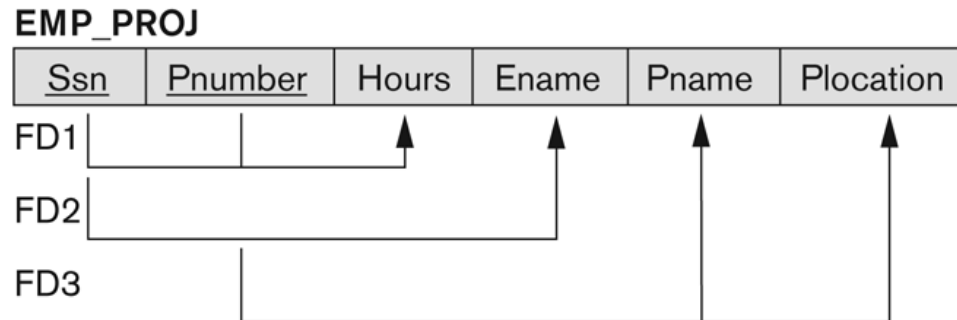
Address (street_address, city, state, zip)

- *street_address, city, state* → *zip*
- *zip* → *city, state*
- *zip, state* → *zip*?
 - This is a trivial FD
 - Trivial FD: $\text{LHS} \supseteq \text{RHS}$
- *zip* → *state, zip*?
 - This is non-trivial, but not completely non-trivial
 - Completely non-trivial FD: $\text{LHS} \cap \text{RHS} = \emptyset$



EXAMPLES OF FD CONSTRAINTS

- If K is a key of R, then K functionally determines all attributes in R



- SSN -> ENAME
- PNUMBER -> {PNAME, PLOCATION}
- {SSN, PNUMBER} -> HOURS



EXAMPLES OF FD CONSTRAINTS

- FD is a property of relational schema R. Must be defined by someone who knows the semantic of attributes of R.
- FD's hold at all times
- Certain FD's can be ruled out based on a given state of the database

TEACH

Teacher	Course	Text
Smith	Data Structures	Bartram
Smith	Data Management	Martin
Hall	Compilers	Hoffman
Brown	Data Structures	Horowitz

- Text $\rightarrow$ course
 - possible FD
- Teacher $\rightarrow$ course
 - ruled out



FUNCTIONAL DEPENDENCIES

- Given a relation R and a set of FD's F
 - Does another FD follow from F ?
 - Are some of the FD's in F redundant (i.e., they follow from the others)?
- Example: **Emp_Dept(ename, ssn, bdate, address, dnumber, dname, dmgrssn)**
 - $F = \{ \text{ssn} \rightarrow \{ \text{ename}, \text{bdate}, \text{address}, \text{dnumber} \}, \text{dnumber} \rightarrow \{ \text{dname}, \text{dmgrssn} \} \}$
 - Additional FD that can be infer from F
 - $\text{ssn} \rightarrow \text{dname}, \text{dmgrssn}$
 - $\text{ssn} \rightarrow \text{ssn}$
 - $\text{dnumber} \rightarrow \text{dname}$



FUNCTIONAL DEPENDENCIES

15.26. Consider the following relation:

A	B	C	TUPLE#
10	b1	c1	1
10	b2	c2	2
11	b4	c1	3
12	b3	c4	4
13	b1	c1	5
14	b3	c4	6

- Which of the following dependencies *may hold* in the above relation?
- If the dependency cannot hold, explain why *by specifying the tuples that cause the violation*.
 - i. $A \rightarrow B$,
 - ii. $B \rightarrow C$,
 - iii. $C \rightarrow B$,
 - iv. $B \rightarrow A$,
 - v. $C \rightarrow A$
- Does the above relation have a potential candidate key? If it does, what is it? If it does not, why not?



CONVERT BUSINESS STATEMENTS INTO DEPENDENCIES.

- Consider the relation

DISK_DRIVE (Serial_number, Manufacturer, Model, Batch, Capacity, Retailer)

- **Example:**

- Disk_drive ('1978619', 'WesternDigital', 'A2235X', '765234', 500, 'CompUSA')

- Write each of the following dependencies as an FD:

- a) The manufacturer and serial number uniquely identifies the drive.
- b) A model number is registered by a manufacturer and therefore can't be used by another manufacturer.
- c) All disk drives in a particular batch are the same model.
- d) All disk drives of a certain model of a particular manufacturer have exactly the same capacity.



Inference Rules for FDs

- Given a set of FDs F , we can **infer** additional FDs that hold whenever the FDs in F hold
- Armstrong's inference rules:
 - IR1. (**Reflexive**)
If $Y \subseteq X$, then $X \rightarrow Y$
 - IR2. (**Augmentation**)
If $X \rightarrow Y$, then $XZ \rightarrow YZ$
(XZ stands for $X \cup Z$)
 - IR3. (**Transitive**)
If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$
- IR1, IR2, IR3 form a **sound** and **complete** set of inference rules



INFERENCE RULES FOR FDS

- Some additional inference rules that are useful:
 - **Decomposition:**
 - If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$
 - **Union:**
 - If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$
 - **Pseudotransitivity:**
 - If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$
- The last three inference rules, as well as any other inference rules, can be deduced from IR1, IR2, and IR3 (completeness property)



PROOFS

- If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$
 - $YZ \rightarrow Y$ (reflexive)
 - $X \rightarrow YZ$ (given)
 - $X \rightarrow Y$ (transitivity)
- If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$
 - $X \rightarrow XY$ (augmenting X in $X \rightarrow Y$)
 - $XY \rightarrow YZ$ (augmenting Y in $X \rightarrow Z$)
 - $X \rightarrow YZ$ (transitivity)
- If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$
 - $WX \rightarrow WY$ (augmenting)
 - $WY \rightarrow Z$ (given)
 - $WX \rightarrow Z$ (transitivity)



INFERENCE RULES FOR FDs

- **Closure** of a set F of FDs is the set F^+ of all FDs that can be inferred from F
- **Closure** of a set of attributes X with respect to F is the set X^+ of all attributes that are functionally determined by X
- X^+ can be calculated by repeatedly applying IR1, IR2, IR3 using the FDs in F



ALGORITHM FOR COMPUTING ATTRIBUTE CLOSURE

- The closure of X (denoted X^+) with respect to F is the set of all attributes functionally determined by X
- **Algorithm for computing the closure**

```
X+ = X
  repeat
    OldX+ = X+
    for each functional dependency Y → Z in F do
      if Y ⊆ X+ then X+ := X+ ∪ Z
  until (X+ = OldX+)
```



EXAMPLE 1: CLOSURE ALGORITHM

Consider following schema

EMP_PROJ(Ssn, Ename, Pno, Pname, Hours)

And FDs

- $F = \{Ssn \rightarrow Ename,$
 $Pno \rightarrow \{Pname, Plocation\},$
 $\{Ssn, Pno\} \rightarrow Hours \}$

- Find the following Closure Sets
- $\{Ssn\}^+ =$
- $\{Pno\}^+ =$
- $\{Ssn, Pno\}^+ =$



EXAMPLE 2: CLOSURE ALGORITHM

StudentGrade (SID, name, email, CID, grade)

F includes:

- $SID \rightarrow name, email$
- $email \rightarrow SID$
- $SID, CID \rightarrow grade$
- $\{CID, email\}^+ = ?$
- $X^+ = \{CID, email\}$

Consider $email \rightarrow SID$

- $X^+ = \{SID, CID, email\}$

Consider $SID \rightarrow name, email$

- $X^+ = \{SID, CID, email, name\}$

Consider $SID, CID \rightarrow grade$

- $X^+ = \{SID, CID, email, name, grade\}$



USING RULES OF FD'S

- Given a relation R and set of FD's F
 - Does another FD $X \rightarrow Y$ follow from F ?
 - Use the rules to come up with a proof
- Example F includes:
 - $SID \rightarrow name, email; email \rightarrow SID; SID, CID \rightarrow grade$
 - **$CID, email \rightarrow grade?$**
 - $email \rightarrow SID$ (given in F)
 - $CID, email \rightarrow CID, SID$ (augmentation)
 - $SID, CID \rightarrow grade$ (given in F)
 - $CID, email \rightarrow grade$ (transitivity)



Minimal Sets of FDs

- A set of FDs is **minimal** if it satisfies the following conditions:
 1. Every dependency in F has a single attribute for its RHS.
 2. We cannot replace any dependency $X \rightarrow A$ in F with $Y \rightarrow A$, $Y \subseteq X$ and still have a set of dependencies that is equivalent to F .
 3. We cannot remove any dependency from F and have a set of dependencies that is equivalent to F .
- Every set of FDs has an equivalent minimal set
- There can be several equivalent minimal sets



EXAMPLE: MINIMAL COVER

- $F = \{ AB \rightarrow C, C \rightarrow A, BC \rightarrow D, ACD \rightarrow B, D \rightarrow EG, BE \rightarrow C, CG \rightarrow BD, CE \rightarrow AG \}$
- Step1: Canonical Form
- Step 2: We need to look for redundant attributes on the LHS
- Step3: We need to look for FDs that are redundant



EXAMPLE: MINIMAL COVER

- For STEP 2: apply either of the rules below to find redundant attributes on the LHS:
 - 1. In an FD $\{X \rightarrow A\}$ from set F , attribute $B \in X$ is redundant in X if F logically implies
$$(F - \{X \rightarrow A\}) \cup \{(X - B) \rightarrow A\}.$$
 - 2. To test if attribute $B \in X$ is redundant in X
 - Step 1: compute $(\{X\} - B)^+$ using the dependencies in F
 - Step 2: check that $(\{X\} - B)^+$ contains B ; if it does, B is redundant



EXAMPLE: MINIMAL COVER

- Consider $ACD \rightarrow B$ to see if any of the three attributes on the LHS is redundant
- Computing $CD^+ = \{ACDEGB\}$
 - Closure contains A and thus, A is redundant.
 - The closure contains B, which tells us that $CD \rightarrow B$ holds.
- Alternatively, we need to prove that F logically implies $CD \rightarrow B$ in place of $ACD \rightarrow B$.
 - $C \rightarrow A$ (given in F)
 - $CD \rightarrow ACD$ (augment with CD)
 - $ACD \rightarrow B$ (given in F)
 - $CD \rightarrow B$ (transitivity)



EXAMPLE: MINIMAL COVER

- Step3: Look for redundant FDs in set F.
 - $CE \rightarrow A$
 - $CG \rightarrow B$
 - (as $CG \rightarrow D$, and $CD \rightarrow B$ in F.)
 - No more redundant FDs in F.
- **Minimal Cover : $\{AB \rightarrow C, C \rightarrow A, BC \rightarrow D, CD \rightarrow B, D \rightarrow E, D \rightarrow G, BE \rightarrow C, CG \rightarrow D, CE \rightarrow G\}$**



EXAMPLE: MINIMAL COVER

- Minimal Cover 2 : { $AB \rightarrow C$, $C \rightarrow A$, $BC \rightarrow D$, $D \rightarrow E$, $D \rightarrow G$, $BE \rightarrow C$, $CG \rightarrow B$, $CE \rightarrow G$ }
- Obtained by eliminating redundant FDs,
 - $CE \rightarrow A$,
 - $CG \rightarrow D$, ($CG \rightarrow B$, $BC \rightarrow D$)
 - $CD \rightarrow B$ ($D \rightarrow G$, $CG \rightarrow B$)
- We need one of { $CG \rightarrow D$, $CG \rightarrow B$ } in the minimal cover – can not eliminate both.



ALGORITHM FOR FINDING MINIMAL COVER F FOR A SET OF FDs E

- Set $F := E$
- Replace each FD $X \rightarrow \{A_1, A_2, \dots, A_n\}$ in F by n FDs $X \rightarrow A_1, X \rightarrow A_2, \dots, X \rightarrow A_n$
- For each FD $X \rightarrow A$ in F
 - For each attribute B of X
 - if $\{F - \{X \rightarrow A\} \cup \{(X - B) \rightarrow A\}\} = F$
 - then replace $X \rightarrow A$ with $(X - B) \rightarrow A$ in F
- For each remaining FD $X \rightarrow A$ in F
 - if $\{F - \{X \rightarrow A\}\} = F$
 - then remove $X \rightarrow A$ from F



EXAMPLE 2: COMPUTE MINIMAL SETS OF FDs

Consider set of FDs $E : \{B \rightarrow A, D \rightarrow A, AB \rightarrow D\}$. We have to find the minimum cover of E .

- **Step1:** Canonical form
- **Step 2:** Determine if $AB \rightarrow D$ has any redundant attribute?

Consider $B \rightarrow A$

$BB \rightarrow AB$ (augmenting with B -- IR2)

$B \rightarrow AB$

$AB \rightarrow D$ (given)

We get $B \rightarrow D$ (*transitivity* – IR3)

Hence, $AB \rightarrow D$ may be replaced by $B \rightarrow D$.

- Now $E' : \{B \rightarrow A, D \rightarrow A, B \rightarrow D\}$.



EXAMPLE 2: COMPUTE MINIMAL SETS OF FDS

Consider set of FDs $E : \{B \rightarrow A, D \rightarrow A, AB \rightarrow D\}$. We have to find the minimum cover of E .

- $E' : \{B \rightarrow A, D \rightarrow A, B \rightarrow D\}$.
- **Step 3:** Look for a redundant FD in E'

$$B \rightarrow D$$

$$D \rightarrow A,$$

Thus, $B \rightarrow A$ (transitive rule)

Eliminate $B \rightarrow A$ in E' .

- The minimum cover of E is $\{B \rightarrow D, D \rightarrow A\}$.



MINIMAL COVER

- Given a relation R (A, B, C, D, E, F) and
- a set of FDs $F = \{A \rightarrow BCE, CD \rightarrow EF, E \rightarrow F, B \rightarrow E, AB \rightarrow CF\}$.
- Compute the minimal cover for F



Equivalence of Sets of FDs

- Two sets of FDs F and G are **equivalent** if:
 - Every FD in F can be inferred from G , and
 - Every FD in G can be inferred from F
 - Hence, F and G are equivalent if $F^+ = G^+$
- **Covers:**
 - F **covers** G if every FD in G can be inferred from F
 - (i.e., if $G^+ \subseteq F^+$)
- F and G are equivalent if F covers G and G covers F



EXAMPLE: EQUIVALENCE BETWEEN SETS OF FUNCTIONAL DEPENDENCIES

- Consider two sets of FDs, F and G ,
 - $F = \{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$ and
 - $G = \{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$
- Are F and G equivalent?
- We need to prove that $F^+ = G^+$.
 - To show that F covers G calculate X^+ w.r.t to F for each FD in G , and then see if this F closure contains attribute in G .
 - Similarly compute G covers F .



EXAMPLE: EQUIVALENCE BETWEEN SETS OF FUNCTIONAL DEPENDENCIES

- Use attribute closure to infer all FDs in F using G
 - $F = \{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$
 - $G = \{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$
- Take the attributes from the LHS of FDs in F and compute attribute closure for each using FDs in G:
 - $(A^+ \text{ using } G) = ABCD;$
 - $(B^+ \text{ using } G) = BC;$
 - $(AC^+ \text{ using } G) = ABCD;$
- All FDs in F are inferred using FDs in G.



EXAMPLE: EQUIVALENCE BETWEEN SETS OF FUNCTIONAL DEPENDENCIES

- $F = \{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$
- $G = \{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$
- Now we see if all FDs in G are inferred by F
 - A^+ using $F = ABCD$;
 - B^+ using $F = BC$;
- All FDs in F can be obtained from G and vice versa, so we conclude that F and G are equivalent.



EXAMPLE: EQUIVALENCE BETWEEN SETS OF FUNCTIONAL DEPENDENCIES

- $F = \{A \rightarrow B, A \rightarrow C\}$
- $G = \{A \rightarrow B, B \rightarrow C\}$
- Are F and G equivalent?

- A^+ using $G = ABC$;
- A^+ using $F = ABC$;
- B^+ using $F = B$,
 - This indicates $B \rightarrow C$ in G is not inferred using the FDs from F .

- Thus, F and G are not equivalent, because $B \rightarrow C$ in G is not inferred from the FDs in F .



FIND THE KEY FOR RELATION R

- Consider a relation $R = \{A, B, C, D, E, F, G, H, I, J\}$ and
- The set of FD's {
 - $A, B \rightarrow C$,
 - $A, G \rightarrow D, H$,
 - $B, G \rightarrow E, F$,
 - $A \rightarrow I$,
 - $H \rightarrow J$ }.
- Find the key for Relation R with given FD's.



ALGORITHM: TO FIND A KEY K FOR R GIVEN A SET F OF FD'S

Input: A universal relation R and a set of functional dependencies F on the attributes of R .

1. Set $K := R$.
2. For each attribute A in K {
 compute $(K - A)^+$ with respect to F ;
 If $(K - A)^+$ contains all the attributes in R ,
 then set $K := K - \{A\}$;
}

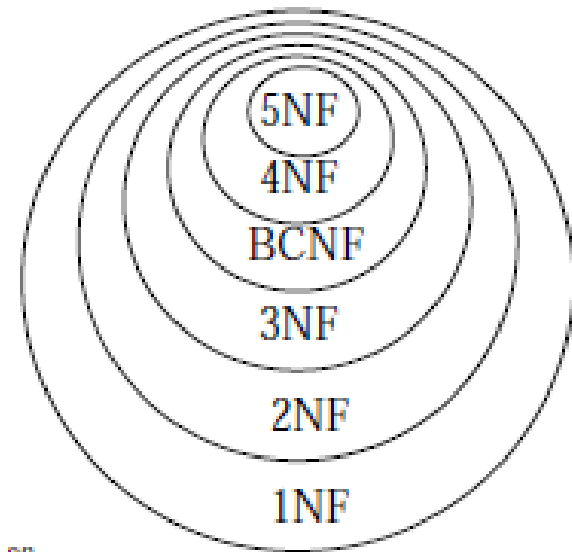


Normalization of Relations

- **Normalization:**

- The process of decomposing "bad" relation by breaking it into smaller relations

- Normalization should maintain properties like lossless join and dependency preservation to ensure a good relational design



NORMAL FORMS

- The database designers don't have to normalize relations to the highest possible normal form (usually 3NF, BCNF or 4NF is enough)
- **Denormalization:**
 - Opposite of normalization
 - Join relations to form a base relation—which is in a lower normal form



SUPERKEYS AND KEYS

- *Superkey*
- *Key*
- *Candidate key*
- *Primary key*
- *Secondary keys*
- **Prime attribute**: it must be a member of *some* candidate key
- **Nonprime attribute**: it is not a member of any candidate key.

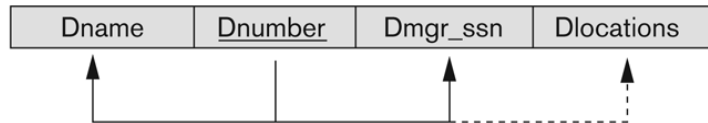


First Normal Form

- It disallows Composite attributes, Multivalued attributes and Nested relations
- Attributes can have atomic values for an *individual tuple*
- It is considered to be a part of the definition of relation

(a)

DEPARTMENT



(b)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

Figure 10.8

Normalization into 1NF.

(a) A relation schema that is not in 1NF. (b)

Example state of relation DEPARTMENT. (c) 1NF version of the same relation with redundancy.

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	<u>Dlocation</u>
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

Normalization of nested relations into 1NF

(a)

EMP_PROJ		Projs	
Ssn	Ename	Pnumber	Hours

(b)

EMP_PROJ			
Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, AliciaJ.	30	30.0
		10	10.0
987987987	Jabbar, Ahmad V.	10	35.0
		30	5.0
987654321	Wallace, Jennifer S.	30	20.0
		20	15.0
888665555	Borg, James E.	20	NULL

(c)

EMP_PROJ1	
Ssn	Ename

EMP_PROJ2		
Ssn	Pnumber	Hours

Figure 10.9

Normalizing nested relations into 1NF. (a) Schema of the EMP_PROJ relation with a *nested relation* attribute PROJS. (b) Example extension of the EMP_PROJ relation showing nested relations within each tuple. (c) Decomposition of EMP_PROJ into relations EMP_PROJ1 and EMP_PROJ2 by propagating the primary key.



Second Normal Form

- Uses the concepts of **FDs**
 - **Full functional dependency:** a FD $Y \rightarrow Z$ where removal of any attribute from Y means the FD does not hold any more
- **EMP_PROJ(SSN, Ename, Pno, Pname, Hours)**
 - $\{SSN, Pno\} \rightarrow Hours$ is a full FD
 - since neither $SSN \rightarrow Hours$ nor $Pno \rightarrow Hours$ hold
 - $\{SSN, Pno\} \rightarrow Ename$ is not a full FD, it is called a partial dependency
 - since $SSN \rightarrow Ename$ also holds



SECOND NORMAL FORM

- **Second Normal Form:**

A relation that is in 1NF and every non-prime attribute is fully functionally dependent on the primary key

- We can decompose a relation into 2NF relations via the process of 2NF normalization

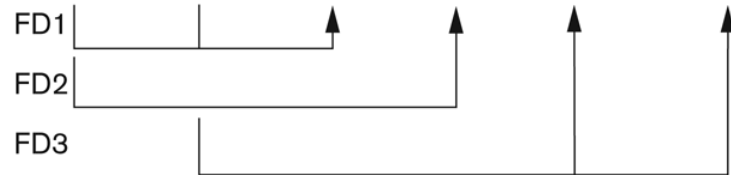


Normalizing into 2NF

(a)

EMP_PROJ

<u>Ssn</u>	<u>Pnumber</u>	Hours	Ename	Pname	Plocation
------------	----------------	-------	-------	-------	-----------



2NF Normalization

EP1

<u>Ssn</u>	<u>Pnumber</u>	Hours
------------	----------------	-------



EP2

<u>Ssn</u>	Ename
------------	-------



EP3

<u>Pnumber</u>	Pname	Plocation
----------------	-------	-----------



Figure 10.10

Normalizing into 2NF and 3NF.
(a) Normalizing EMP_PROJ into 2NF relations. (b) Normalizing EMP_DEPT into 3NF relations.



Third Normal Form

- **Transitive functional dependency:**
 - a FD $X \rightarrow Z$ that can be derived from two FDs $X \rightarrow Y$ and $Y \rightarrow Z$
- **Emp_Dept(SSN, Ename, Bdate, Address, Dno, Dname, Dmgrs_ssn)**
 - $SSN \rightarrow DMGRSSN$ is a **transitive** FD
 - Since $SSN \rightarrow DNUMBER$ and $DNUMBER \rightarrow DMGRSSN$ hold
 - $SSN \rightarrow ENAME$ is **non-transitive**
 - Since there is no set of attributes X where $SSN \rightarrow X$ and $X \rightarrow ENAME$

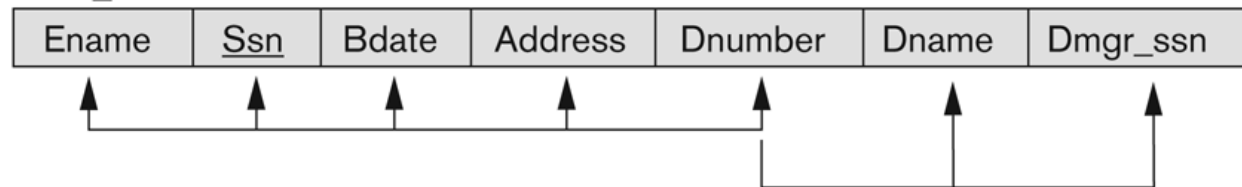


THIRD NORMAL FORM

- Third Normal Form:

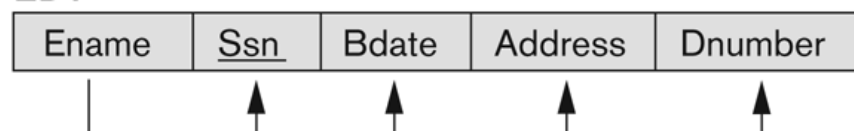
A relation that is in 2NF, and in which no non-prime attribute is transitively dependent on the primary key.

EMP_DEPT

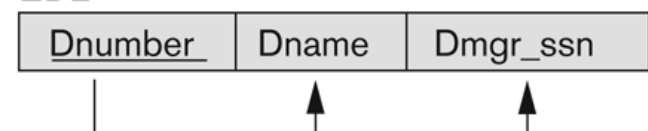


3NF Normalization

ED1



ED2



THIRD NORMAL FORM

○ NOTE:

- In $X \rightarrow Y$ and $Y \rightarrow Z$, with X as the primary key, we consider this a problem only if Y is not a candidate key.
- Consider EMP (SSN, Emp#, Salary).
 - Here, $SSN \rightarrow Emp\# \rightarrow Salary$ and Emp# is a candidate key.
- When Y is a candidate key, there is no problem with the transitive dependency



NORMAL FORMS DEFINED INFORMALLY

- 1st normal form
 - All attributes depend on **the key**
- 2nd normal form
 - All attributes depend on **the whole key**
- 3rd normal form
 - All attributes depend on **nothing but the key**



SUMMARY OF NORMAL FORMS

based on Primary Keys

Table 10.1

Summary of Normal Forms Based on Primary Keys and Corresponding Normalization

Normal Form	Test	Remedy (Normalization)
First (1NF)	Relation should have no multivalued attributes or nested relations.	Form new relations for each multi-valued attribute or nested relation.
Second (2NF)	For relations where primary key contains multiple attributes, no nonkey attribute should be functionally dependent on a part of the primary key.	Decompose and set up a new relation for each partial key with its dependent attribute(s). Make sure to keep a relation with the original primary key and any attributes that are fully functionally dependent on it.
Third (3NF)	Relation should not have a nonkey attribute functionally determined by another nonkey attribute (or by a set of nonkey attributes). That is, there should be no transitive dependency of a nonkey attribute on the primary key.	Decompose and set up a relation that includes the nonkey attribute(s) that functionally determine(s) other nonkey attribute(s).

GENERAL NORMAL FORM DEFINITIONS (FOR MULTIPLE KEYS)

- The above definitions consider the primary key only
- The following more general definitions take into account relations with multiple candidate keys
- A relation schema R is in **2NF** if every non-prime attribute A in R is fully functionally dependent on *every* key of R

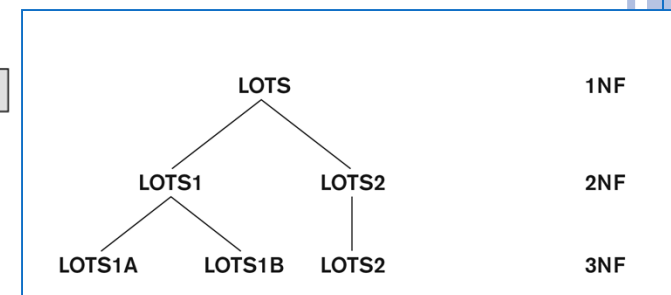
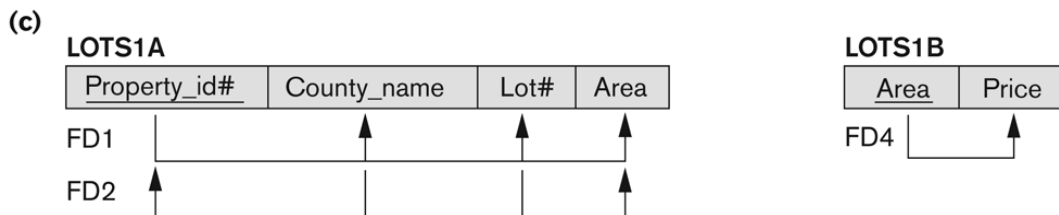
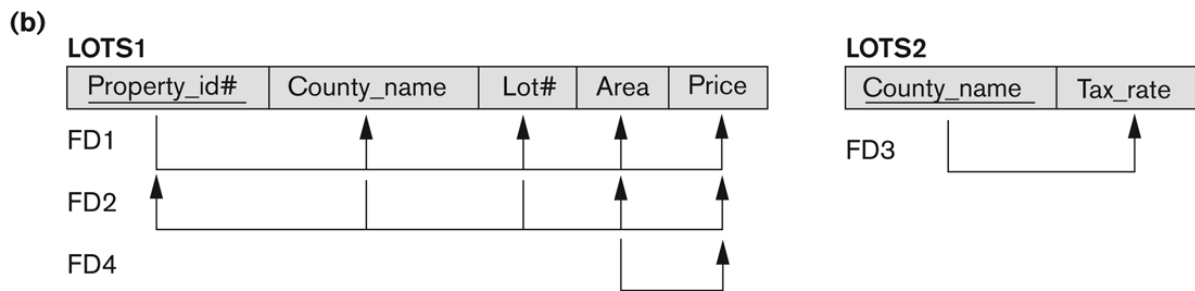
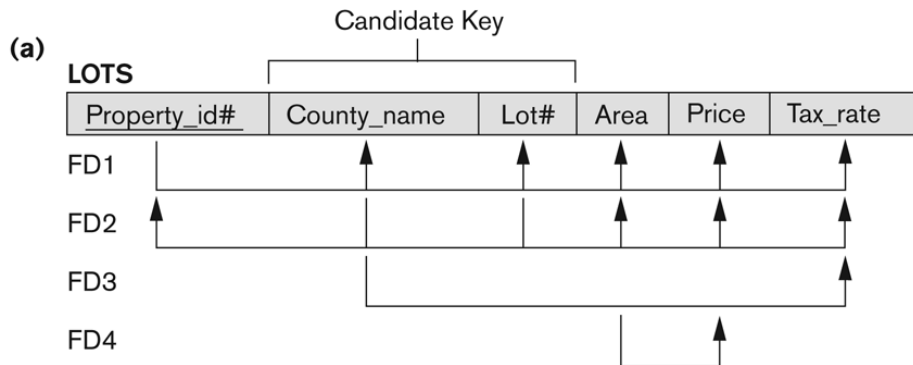


GENERAL NORMAL FORM DEFINITIONS

- A relation R is in **3NF** if whenever a FD $X \rightarrow A$ holds in R , then either:
 - a) X is a superkey of R , or
 - b) A is a prime attribute of R



Successive Normalization of LOTS into 2NF and 3NF



BCNF (Boyce-Codd Normal Form)

- A relation schema R is in **Boyce-Codd Normal Form (BCNF)** if whenever an **FD** $X \rightarrow A$ holds in R , then **X is a superkey** of R
- Each normal form is strictly stronger than the previous one
 - Every 2NF relation is in 1NF
 - Every 3NF relation is in 2NF
 - Every BCNF relation is in 3NF
- There exist relations that are in 3NF but not in BCNF
- The goal is to have each relation in BCNF (or 3NF)



Boyce-Codd Normal Form

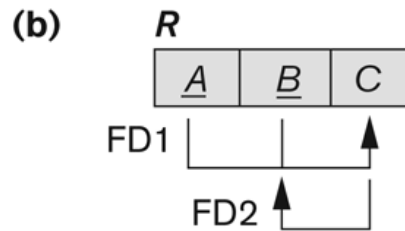
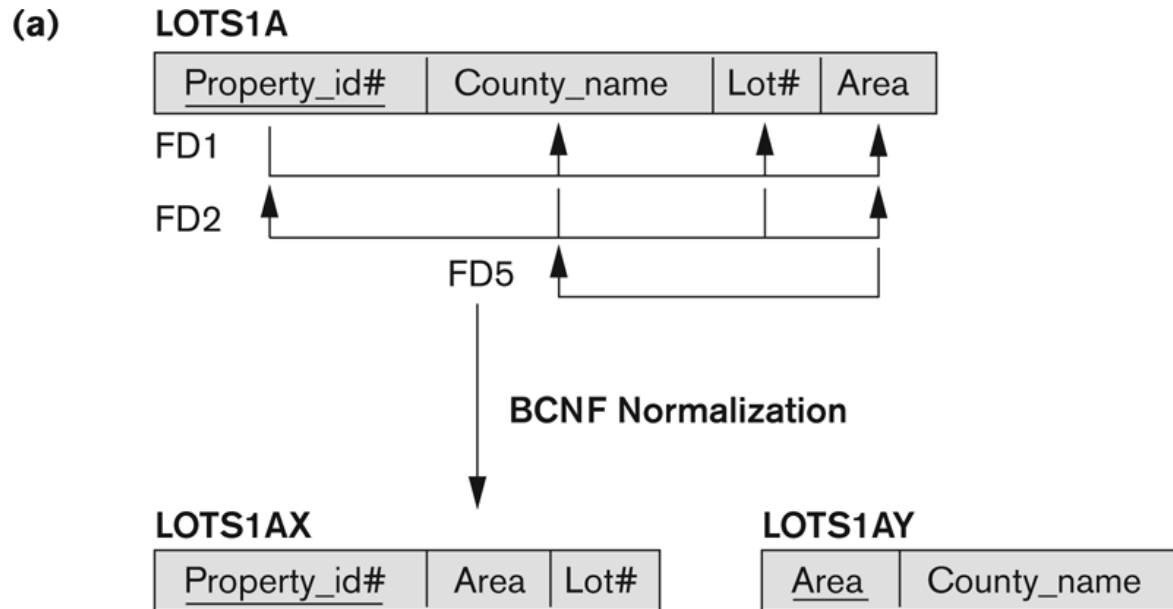


Figure 10.12

Boyce-Codd normal form. (a) BCNF normalization of LOTS1A with the functional dependency FD2 being lost in the decomposition. (b) A schematic relation with FDs; it is in 3NF, but not in BCNF.

A relation that is in 3NF but not in BCNF

Two FDs exist in the relation TEACH:

FD1: { student, course} -> instructor

FD2: instructor -> course

{student, course} is a
candidate key

TEACH

Student	Course	Instructor
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe
Narayan	Operating Systems	Ammar

Figure 10.13

A relation TEACH that
is in 3NF but not
BCNF.



A relation that is in 3NF but not in BCNF

Two FDs exist in the relation TEACH:

FD1: { student, course} $\rightarrow$ instructor

FD2: instructor $\rightarrow$ course

Three possible decompositions for relation TEACH

{student, instructor} and {student, course}

{course, instructor} and {student, course}

{course, instructor} and {student, instructor}

{student, course} is a
candidate key

All three decompositions lose FD1. We sacrifice the FD preservation. But cannot sacrifice the non-additivity property after decomposition.

Only the 3rd decomposition will not generate spurious tuples after join

TEACH

Student	Course	Instructor
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe
Narayan	Operating Systems	Ammar

EXAMPLE

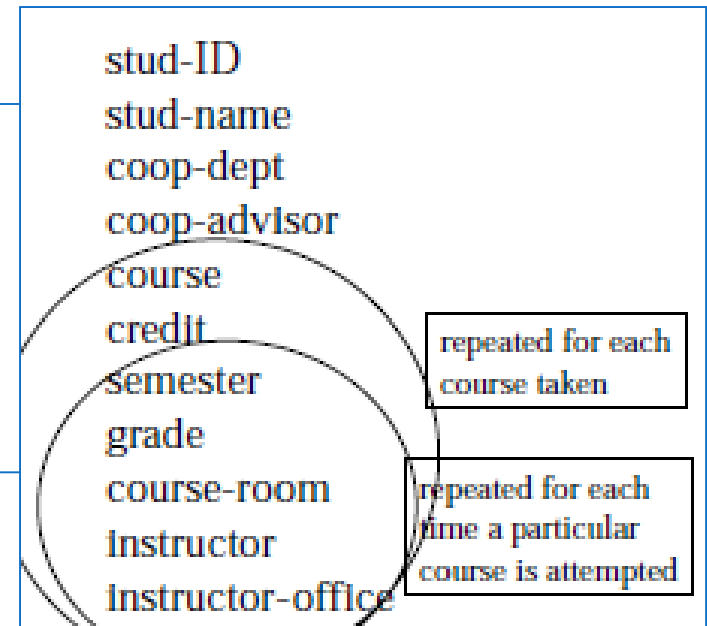
- What is the key for R ? Given
- $F = \{A, B \rightarrow C, B, D \rightarrow E, F, A, D \rightarrow G, H, A \rightarrow I, H \rightarrow J\}$
- Key
 - ABD
- 2NF
 - R1 (A, B, C) R2 (B, D, E, F) R3 (A, D, G, H, J) R4 (A, I), R=(ABD)
- 3NF
 - R1 (A, B, C) R2 (B, D, E, F) R3.1 (A, D, G, H) R3.2 (H, J) R4 (A, I), R=(ABD)

EXAMPLE: DESIGNING A STUDENT DATABASE

1. Students take courses
2. Students typically take more than one course
3. Students can fail courses and can repeat the same course in different semesters => Students can take the same course more than once.
4. Students are assigned a grade for each course they take.

The data that needs to be retained has repeating groups

Procedure: Remove repeating groups by adding extra rows to hold the repeated attributes. => Add redundancy.



DESIGNING A STUDENT DATABASE -2

- **Student_course(sID, sname, dept, advisor, course, credit, semester, grade, course-room, instructor, instructor-office)**
- 1NF: Tables should have no repeating groups
- Problems:
 - Redundancy
 - Insert anomalies
 - Delete anomalies
 - Update problems



DESIGNING A STUDENT DATABASE -3

- *sname, dept, advisor are dependent only upon sID*
- *credit dependent only on course and is independent of which semester it is offered and which student is taking it.*
- *course-room, instructor and instructor-office only depend upon the course and the semester (are independent of which student is taking the course).*
- *Only grade is dependent upon all 3 parts of the original key.*
- **2NF: The non-prime attributes should depend on the whole key.**
- **Procedure: remove partial key dependencies**



DESIGNING A STUDENT DATABASE -4

- Student_course(sID, sname, dept, advisor, course, credit, semester, grade, course-room, instructor, instructor-office)
- Student (sID, sname, dept, advisor)
- Student_Reg (sID, course, semester, grade)
- Course (course, credit)
- Course_Offering (course, semester, course-room, instructor, instructor-office)
- 2NF: There are no non-key attributes with partial key dependencies in any table.
- Less redundancy
- Still insert/delete/update problems



DESIGNING A STUDENT DATABASE -5

- **3NF: The non-key attributes should not depend on other non-key attributes.**
- **Procedure: remove non-key dependencies (transitive functional dependencies)**



DESIGNING A STUDENT DATABASE -3NF

- Student (sID, sname, dept, advisor)
 - Student_Reg (sID, course, semester, grade)
 - Course (course, credit)
 - Course_Offering (course, semester, course-room, instructor, instructor-office)
-
- Student (sID, sname, dept, advisor)
 - Student_Reg (sID, course, semester, grade)
 - Course (course, credit)
 - Course_Offering (course, semester, course-room, instructor)
 - Instructor (instructor, instructor-office)
 - More organized, Less redundancy (save space??)
 - performance problems?? (indirect references)



DESIGNING A STUDENT DATABASE -7

- In normalization, we are seeking to make sure that attributes depend:
 - **on the key (1NF)**
 - **on the whole key (2NF)**
 - **on nothing but the key (3NF)**
- We have not checked whether some part of the key itself depends on a non-key attribute(s).



DESIGNING A STUDENT DATABASE -8

- Suppose:

1. each department may have more than one advisor
2. a advisor works for only one department (*advisor* $\rightarrow$ *dept*)
3. a student may be associated with several departmental programs.

- Student (sID, sname, dept, advisor)

- Part of the compound key is determined by a non-key attribute

- *advisor* $\rightarrow$ *dept*

- *sname* is function of only *sID*
- *dept* is function of only *advisor*



DESIGNING A STUDENT DATABASE -9

- Student (sID, sname)
- Advisor (advisor, dept)
- Student-Advice (sID, advisor)
- Student_Reg (sID, course, semester, grade)
- Course (course, credit)
- Course_Offering (course, semester, course-room, instructor)
- Instructor (instructor, instructor-office)

BCNF: Table in 3NF and there are no dependencies of part of the compound key on another attribute



NORMALIZATION EXAMPLE -- SALES ORDER

Sales Order

*Fiction Company
202 N. Main
Manhattan, KS 66502*

CustomerNumber:	1001	Sales Order Number:	405
Customer Name:	ABC Company	Sales Order Date:	2/1/2000
Customer Address:	100 Points	Clerk Number:	210
	Manhattan, KS 66502	Clerk Name:	Martin Lawrence

Item Ordered	Description	Quantity	Unit Price	Total
800	widgit small	40	60.00	2,400.00
801	angimajigger	20	20.00	400.00
805	thingibob	10	100.00	1,000.00

Order Total

3,800.00

Fields in the original data table will be as follows:

SalesOrderNo,
Date,
CustomerNo,
CustomerName,
CustomerAdd,
ClerkNo,
ClerkName,
ItemNo,
Description,
Qty, UnitPrice

NORMALIZATION: FIRST NORMAL FORM

- Separate Repeating Groups into New Tables.
- ***Repeating Groups*** Fields that may be repeated several times for one document/entity
- The primary key of the new table (repeating group) is always a composite key;
- Relations in 1NF:
 - SalesOrderNo, ItemNo, Description, Qty, UnitPrice
 - SalesOrderNo, Date, CustomerNo, CustomerName, CustomerAdd, ClerkNo, ClerkName



NORMALIZATION: SECOND NORMAL FORM

- Remove Partial Dependencies.
- Note: Do not treat price as dependent on item. Price may be different for different sales orders (discounts, special customers, etc.)
- Relations in 2NF
 - ItemNo, Description
 - SalesOrderNo, ItemNo, Qty, UnitPrice
 - SalesOrderNo, Date, CustomerNo, CustomerName, CustomerAdd, ClerkNo, ClerkName



NORMALIZATION: SECOND NORMAL FORM

- What if we did not Normalize the Database to Second Normal Form?
 - Redundancy
 - Delete Anomalies
 - Insert Anomalies
 - Update Anomalies



NORMALIZATION: THIRD NORMAL FORM

- Remove transitive dependencies
- **Relations in 3NF**
 - Customers: CustomerNo, CustomerName, CustomerAdd
 - Clerks: ClerkNo, ClerkName
 - Inventory Items: *ItemNo*, Description
 - Sales Orders: SalesOrderNo, Date, CustomerNo, ClerkNo
 - SalesOrderDetail: SalesOrderNo, ItemNo, Qty, UnitPrice



NORMALIZATION: THIRD NORMAL FORM

- **What if we did not Normalize the Database to Third Normal Form?**
 - Redundancy – Detail for Cust/Clerk would appear on every SO
 - Delete Anomalies – Delete a sales order, delete the customer/clerk
 - Insert Anomalies – To insert a customer/clerk, must insert sales order.
 - Update Anomalies – To change the name/address, etc, must change it on every SO.



CHAPTER SUMMARY

- Informal Design Guidelines for Relational Databases
- Functional Dependencies (FDs)
 - Definition, Inference Rules, Equivalence of Sets of FDs, Minimal Sets of FDs
- Normal Forms Based on Primary Keys
- General Normal Form Definitions (For Multiple Keys)
- BCNF (Boyce-Codd Normal Form)

